

Programação Orientada a Objetos 2024/2025

Exercícios

Ficha Nº 6

Agregação Agregação vs. Composição

Introdução teórica aos conceitos usados nesta ficha de exercícios

Agregação

Trata-se de uma forma de associação na qual um objeto da classe *A* “**vê**” um ou mais objetos da classe *B*. Significa que os objetos de *A* de alguma forma usam/vêm/atuam sobre os objetos da classe *B*. No entanto, *A* não é “dono” dos objetos de *B*. Se o objeto da classe *A* desaparecer, os objetos de *B* que participavam na associação continuam a existir: Os objetos de *A* não controlam nem são responsáveis pela existência dos objetos de *B*. Neste aspeto, a agregação contrasta com a associação do tipo de composição visto na ficha anterior.

- Quando os objetos de *A* são criados, os de *B* já existirão;
- Quando os objetos de *A* são destruídos, os de *B* podem permanecer
- Na atribuição e cópia, poderá ser permissível os objetos de *B* serem partilhados entre ambos o original e cópia de *A*.

Aspetos de implementação.

- Dado que os objetos de *B* têm uma existência independente dos objetos de *A*, na classe *A* os objetos de *B* apenas podem ser referidos (por referência ou ponteiro), ou seja, não será possível ter os objetos de *B* fisicamente contidos dentro dos objetos de *A*, caso contrário, quando o objeto de *A* desaparece, os objetos de *B* nele contidos também desapareceriam, contrariando a lógica da agregação.
- Se os objetos de *B* referidos num objeto de *A* desaparecerem, o objeto de *A* deve ter a capacidade de ser avisado desse facto, de forma a não voltar a usar esses objetos de *B* referidos, que agora já não existem. Isto normalmente envolve mecanismos explicitamente criados pelo programador, dado que o compilador não faz nada relativamente a esta situação.

Exemplo: classe *Turma* e classe *Aluno*. Um objeto *Turma* tem (agrega) um conjunto de objetos de *Aluno*. Os objetos de *Aluno* envolvidos já existiam antes e continuam a existir mesmo que o objeto *Turma* que os agrega desapareça.

1. (Opcional) Pessoa – Classe auxiliar

Considere uma classe *Pessoa* tendo como atributos nome, número de bilhete de identidade (BI), e número de contribuinte (NIF). Um objeto de *Pessoa* só pode ser criado se lhe forem indicados todos os seus dados. A classe *Pessoa* deve ter funções membros que permitam obter cada um dos seus dados, atualizar o nome (apenas o nome), e obter uma *string* com a descrição do seu conteúdo. Esta classe vai ser necessária para os exercícios seguintes. Pode fazer esta classe se sentir necessidade ou usar o código dado abaixo.

```
class Pessoa {
public:
    Pessoa(const string & nome, int nBI, int nNIF)
        : nome(nome), BI(nBI), NIF(nNIF) {}

    std::string getNome() const { return nome; }

    int getNumeroBI() const { return BI; }

    int getNumeroNIF() const { return NIF; }

    void atualizarNome(const string& novoNome) {
        nome = novoNome;
    }

    string descricao() const {
        std::ostringstream oss;
        oss << "Nome: " << nome
            << ", BI: " << BI
            << ", NIF: " << NIF;
        return oss.str();
    }
private:
    string nome;
    int BI;
    int NIF;
};
```

2. Registo Civil de pessoas

Pretende-se uma classe **RegistoCivil** que representa a o conjunto de pessoas existentes num determinado país. Um objeto desta classe tem as seguintes características

- Quando construído, exige o nome do país a que diz respeito. Inicialmente não tem pessoa nenhuma.
- É possível obter, mas não mudar o nome do país (isto não significa necessariamente o uso de *const*).
- Permite adicionar uma nova pessoa, **a qual é construída no contexto da classe e não fora dela**. Para esta operação são indicados os dados da nova pessoa. A operação falha se já existir nesse país uma pessoa com o BI indicado, ou se já tiver atingido o número máximo de pessoas.
- Permite adicionar pessoas lidas de um ficheiro de texto (nome do ficheiro é indicado). O ficheiro tem uma pessoa por linha, no formato nome BI NIF. O nome tem apenas uma palavra.
- Permite apagar uma pessoa dado o seu BI.
- Permite obter o nome de uma pessoa dado o seu BI.
- Permite obter a listagem de todas as pessoas numa string, uma pessoa por linha.
- Permite atualizar o nome de uma pessoa dado o seu BI e o novo nome.
- Permite obter o número de pessoas atualmente existentes no país.

a) Identifique o **tipo de associação** existente entre a classe **RegistoCivil** e a classe **Pessoa**. Trata-se de Agregação ou de Composição?

b) Assuma a seguintes restrições:

Não pode usar memória dinâmica nem pode usar *coleções standard* (como por exemplo, *vector*, quem nem foram ainda abordados). Pode assumir que haverá no máximo 50 pessoas. **Pode** modificar a classe pessoa de forma a ser possível construir objetos pessoa sem informação nenhuma, sendo que o BI fica inicializado a -1, que significa que este objeto pessoa não representa pessoa nenhuma. Se recorrer a esta alteração tem de a justificar ao professor.

Explique os dados da classe e a estratégia que seria implementada (para debater na aula).

c) **TPC**: implemente a classe considerando as restrições de b)

d) Assuma agora as seguintes restrições:

- **Não sabe** à partida quantas pessoas vão existir e **não pode** mexer na classe Pessoa que foi fornecida.
- Continua a não poder usar *coleções standard* (como por exemplo, *vector*). **Pode** usar memória dinâmica se isso o ajudar.

Estas restrições apontam para o uso de memória dinâmica:

Mais especificamente -> **um array dinâmico de pessoas.**

No entanto:

- **O resto do programa poderá precisar de saber que as pessoas existem.** Por exemplo, pode haver um clube de futebol que registe ponteiros para as pessoas (solução perigosa e falível). Havendo um ponteiro para uma pessoa algures no programa, é importante garantir que esse objeto pessoa não muda para outro local em memória. No entanto, se os objetos Pessoa forem armazenados num *array* dinâmico, a criação (ou eliminação) de uma pessoa obriga à realocação de todo o *array*, ou seja, as pessoas mudam de lugar em memória e o ponteiro no clube (por exemplo) torna-se inválido

Assim:

- Considere usar um ***array* dinâmico de ponteiros para objeto Pessoa**. Aquilo que é realocado é o *array* de ponteiros, permanecendo os objetos pessoas no mesmo lugar que já ocupavam.
- Esta solução é mais complexa, mas mais robusta.
- **Vantagem adicional desta solução:** não aloca *array* de objetos Pessoa – aloca um *array* de ponteiros para Pessoa, e **cada objeto Pessoa é alocado individualmente**, um por um, o que **remove a restrição de ter de haver um construtor por omissão para a Pessoa**.

Implemente a classe com estas restrições e considerações. Verifique que não se esqueceu de função nenhuma.

Organize o seu código em *.h* e *.cpp*. Use uma função *main* para testar a classe.

No final do exercício deverá ter

- Consolidado os conceitos e sintaxe de composição objetos com e sem memória dinâmica
- Ter experimentado uma situação de gestão de recursos dinâmicos já com alguma complexidade

3. Clube de futebol

Pretende-se representar o conceito de *clube de futebol* em C++. Um clube é constituído por um conjunto de pessoas que jogam à bola nos feriados. A classe pretendida chama-se *Clube* e deve cumprir a seguinte funcionalidade:

- Tem um nome, e um conjunto de 11 jogadores (pessoas). Não interessa a posição em que jogam (todos são guarda-redes, se for preciso)
- Um objeto de Clube só pode ser criado mediante a indicação do seu nome. Inicialmente não existem jogadores.
- Deve ser possível acrescentar um jogador ao clube. O jogador é uma pessoa previamente existente. A mesma pessoa não pode pertencer ao mesmo Clube mais que uma vez em simultâneo, mas **pode pertencer a vários clubes diferentes. Qualquer alteração na pessoa (dados dela) será visível em todos os clubes em que essa pessoa participe.**

Dica: Estas informações são muito relevantes para perceber a relação entre *Clube* e *Pessoa*, e também para perceber como é que as pessoas são armazenadas e quais os parâmetros da função para adicionar sócios.

- Não se pretende ter meia dúzia de objetos pessoa semeados na função main(). Existe uma classe para gerir as Pessoas e essa classe é o **RegistoCivil do exercício anterior**. **Os sócios do clube são pessoas geridas pelo RegistoCivil**.

Dica: Esta informação é relevante para perceber se é necessário adicionar alguma funcionalidade ao RegistoCivil do exercício anterior.

- Deve ser possível saber se uma pessoa é sócia do clube. Para tal é indicado um BI.
- Deve ser possível remover um sócio do clube dado o seu bilhete de identidade (provavelmente não pagou a quota).
- Deve ser possível listar os dados de todos os sócios de um clube.

- a) Identifique o tipo de relação que existe entre os objetos de Sócio e os objetos de Clube. Este assunto deve ser debatido com o professor (participando na aula).

Esta é uma alínea de planeamento, e o assunto deve ser debatido com o professor participando na aula.

- b) Verifique se é necessário o Clube saber que o registo civil existe. Em caso afirmativo, estando o RegistoCivil envolvido nesta questão, explique como se relaciona o Clube com o RegistoCivil (tipo de associação), e como se implementa na classe Clube o conhecimento acerca da existência do RegistoCivil.

Esta é uma alínea de planeamento, e o assunto deve ser debatido com o professor participando na aula. Nota: existem duas respostas possíveis e devem ser ambas debatidas.

- c) Explique como pode fazer para lidar com a seguinte situação: havia uma pessoa (no registo civil). Essa pessoa fez-se sócia do clube. Posteriormente a pessoa deixou de existir. Sendo assim:

- i) Qual a classe que trata da existência ou não existência da Pessoa?
- ii) A pessoa continua a existir no RegistoCivil?
- iii) O que acontece / O que deve acontecer na classe Clube?

Considerações para as alíneas seguintes: sendo as pessoas a usar no **Clube** geridas pelo **RegistoCivil** do exercício anterior, existem apenas duas grandes alternativas de implementação:

- A.** O clube sabe que o RegistoCivil existe e dirige-se a este para obter dados sobre os sócios. Precisa apenas de armazenar os BI dos sócios e o conhecimento que o RegistoCivil existe. É muito fácil saber se um BI deixou de ser válido porque o RegistoCivil intermedeia todos os acessos à Pessoa por parte do Clube. Se a pessoa deixar de existir, o Clube identificará essa situação analisando a resposta que o RegistoCivil lhe retorna, removendo o sócio que deixou de existir.

Esta solução faz lembrar um cenário chave forasteira – chave primária de bases de dados relacionais. É fácil de implementar, mas não é a mais eficiente, pois qualquer consulta aos dados do sócio implica o reencaminhamento dessa consulta ao RegistoCivil, sendo-lhe dado o BI (por exemplo), obrigando a uma consulta dentro do RegistoCivil. O Clube precisa de saber que o RegistoCivil existe.

B. O clube armazena “referências” (referencias/ponteiros) para as pessoas. O Clube tem acesso direto aos objetos pessoa, não precisando de se dirigir ao RegistoCivil. Não precisa de saber que o RegistoCivil existe. O Clube fica fragilizado na medida em que a pessoa pode deixar de existir sem que o clube se aperceba e as referências/ponteiros para as pessoas (sócios) armazenadas no clube podem ficar inválidas a qualquer momento e sem que o Clube se possa proteger.

Esta solução também é fácil de implementar, e é a mais eficiente, pois dispensa “consultas por BI” a cada acesso a dados dos sócios. No entanto tema fragilidade descrita atrás que só se consegue resolver de forma eficaz com lógica adicional (por exemplo, *smartpointers*).

Em ambas as estratégias, a sua função *main* conterá um objeto RegistoCivil, um ou mais objetos Clube, mas nenhum objeto pessoa.

d) Implemente a classe **Clube** através da estratégia **A**.

e) Implemente a classe **Clube** através da estratégia **B**. **Identifique** os métodos de consulta na **classe RegistoCivil** que se tornam necessário (para uso do Clube) e acrescente-os.

Nesta estratégia, o clube regista referências ou ponteiro. Verifique se são ambos possíveis. Se um deles não for, explique porquê.

Garanta que os dados da pessoa só podem ser alterados através do RegistoCivil.

No final deste exercício é suposto

- Compreender os conceitos de composição e agregação de objetos, incluindo as diferenças entre os dois conceitos e os cenários em que se deve aplicar um ou o outro.
- Compreender a forma de escolher referências vs. ponteiros.
- Compreender a implementação de agregação através de referências ou ponteiros.